

LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY SIERRA LEONE

FACULTY OF INFORMATION & COMMUNICATIONS TECHNOLOGY

PROG103 - PRINCIPLE OF STRUCTURED PROGRAMMING

FINAL PROJECT DOCUMENTATION

Project Title: Student Academic Portal Management System
Application Type: GUI-Based Python Application

Course	Principle of Structured Programming
System Area	Education System - Student Academic Portal
Language	Python
GUI Library	CustomTkinter / Tkinter
Storage	JSON File Storage
Submission	Source Code, Documentation, README, License, Screenshots

Prepared By: _____

Group Name: _____

Submission Date: _____

Table of Contents

1. Introduction
2. Project Rationale
3. Problem Description
4. Aim and Objectives
5. SDG Relevance
6. Scope of the System
7. System Requirements
8. System Analysis
9. System Design
10. Algorithm and Flowchart
11. Structured Programming Application
12. Functional Requirements
13. Non-Functional Requirements
14. Data Privacy and Ethics
15. Testing and Results
16. Screenshots
17. GitHub and Open-Source Evidence
18. Installation and User Guide
19. Challenges Encountered
20. Conclusion and Recommendations
21. References
22. Appendix: Key Source Code Explanation

1. Introduction

The Student Academic Portal Management System is a GUI-based Python application developed for the PROG103 Principle of Structured Programming final project. The system is based on the education system category because it solves a real academic management problem faced by students: difficulty in organizing courses, assignments, grades, messages, and academic schedules in one place.

The application was developed using Python with CustomTkinter. It demonstrates structured programming principles by using meaningful variables, lists, dictionaries, decision structures, loops, file handling, user-defined functions, and modular code organization. The system allows demo accounts to use the default Limkokwing sample subjects, while newly registered students can enter their own subjects during signup. Those subjects are then used throughout the student session for courses, assignments, calendar events, grades, and lecturer messages.

2. Project Rationale

The project brief requires a graphical user interface system that addresses a real-world problem in Sierra Leone, aligns with at least one Sustainable Development Goal, and applies structured programming concepts. This project solves an education management problem by giving students a digital academic portal that centralizes personal academic information.

Many students in Sierra Leone still depend on handwritten notes, scattered WhatsApp messages, printed timetables, and informal reminders to track their school work. This can lead to missed assignments, poor planning, and lack of access to clear academic records. The Student Academic Portal improves this by providing a user-friendly desktop application that stores student information and displays academic modules clearly.

3. Problem Description

The identified problem is poor access to organized student academic information. Students often need to check different places to know their subjects, assignment deadlines, grades, calendar activities, and messages from lecturers. Without a central digital tool, students may forget deadlines or fail to monitor their academic progress.

The proposed system provides a centralized portal that handles student registration, login, subject entry, course display, assignment display, grade tracking, calendar events, and lecturer messages. This makes academic information easier to access and improves student planning.

4. Aim and Objectives

Aim

To develop a GUI-based Student Academic Portal Management System using Python that helps students organize their academic activities and demonstrates structured programming principles.

Specific Objectives

- Allow students to create an account through a signup form.
- Generate a unique student ID when the student does not enter one.
- Allow newly registered students to enter their own subjects during signup.

- Display student-specific subjects throughout the dashboard, courses, assignments, grades, calendar, and messages.
- Keep demo accounts working with the original default Limkokwing courses.
- Validate user input to prevent empty names, invalid email addresses, missing subjects, weak passwords, and mismatched passwords.
- Store student records permanently in a JSON file.
- Provide a clean and user-friendly graphical interface.
- Demonstrate variables, constants, data types, lists, dictionaries, functions, if/elif/else decisions, loops, and file handling.

5. SDG Relevance

This project is aligned with Sustainable Development Goal 4: Quality Education. SDG 4 focuses on inclusive and equitable quality education and lifelong learning opportunities. The portal supports this goal by improving academic organization and helping students access learning information in one system.

The system also supports digital transformation in education in Sierra Leone. By allowing students to track subjects, assignments, grades, and academic messages, the project encourages better learning management and student responsibility.

6. Scope of the System

The system focuses on student academic management. It does not replace a complete university management system, but it provides important academic functions needed by students.

Included Features

- Student signup and login
- Automatic student ID generation
- Student subject entry during signup
- Dashboard overview
- Courses page
- Assignments page
- Grades page
- Calendar page
- Messages page
- Settings page
- JSON-based record storage

Excluded Features

- Online deployment
- Real university database integration
- Payment processing
- Administrative staff dashboard
- Lecturer upload dashboard

7. System Requirements

Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM recommended
- Keyboard and mouse
- Screen resolution of at least 900 x 600

Software Requirements

- Python 3.10 or later
- CustomTkinter library
- Matplotlib library
- Internet connection only when email sending is enabled
- Any Python IDE such as VS Code, PyCharm, or IDLE

8. System Analysis

Input Module

The input module collects student information from the signup and login forms. Data collected includes first name, last name, email address, optional student ID, institution, subjects, password, and confirm password.

Processing Module

The processing module validates the entered data, generates IDs, parses subjects, stores data, authenticates login, and builds user-specific academic data. It uses decision structures such as if statements to check missing fields and loops to process subject lists.

Output Module

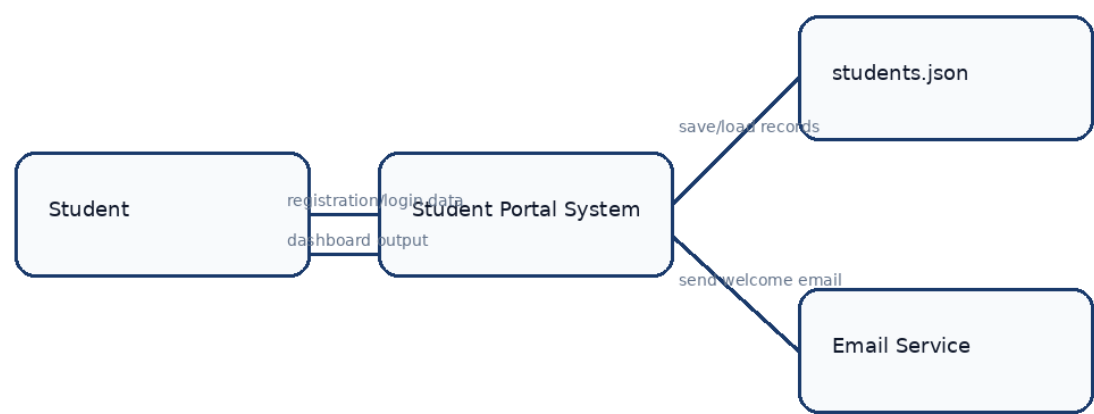
The output module displays a dashboard, course cards, assignments, grades, calendar events, and messages. It also displays error messages and success messages using GUI message boxes.

Storage Module

Student records are stored in a JSON file named students.json. This allows new registered students to log in again after the application is closed and reopened.

9. System Design

Context Diagram



The context diagram shows the student interacting with the Student Portal System. The system saves and loads records from students.json and optionally sends welcome emails through an email service.

Main Modules

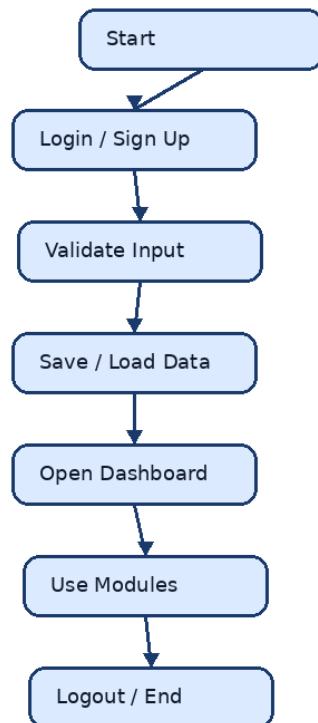
Module	Purpose	Main Functions Used
Registration	Creates new student accounts and saves subjects	signup_submit(), parse_subjects(), generate_student_id(), save_students()
Login	Authenticates student ID and password	login(), load_students()
Dashboard	Shows GPA, active courses, pending tasks, completed work	render_dashboard(), get_user_courses(), get_user_assignments()
Courses	Displays demo or user-entered subjects	render_courses(), build_courses_from_subjects()
Assignments	Generates assignment list based on subjects	get_user_assignments(), render_assignments()
Calendar	Shows academic dates based on user subjects	get_user_events(), render_calendar()
Messages	Creates lecturer messages for selected subjects	get_user_conversations(), render_messages()

10. Algorithm and Flowchart

General Algorithm

1. Start the application.
2. Load existing students from students.json.
3. Display the login page.
4. If the user selects signup, collect personal details and subjects.
5. Validate all required fields.
6. If no student ID is entered, generate a unique student ID.
7. Save the student record in the JSON file.
8. Return the user to the login page.
9. When a student logs in, compare the entered ID and password with stored records.
10. If login is correct, open the dashboard.
11. If the account has custom subjects, generate courses, assignments, calendar, grades, and messages from those subjects.
12. If the account is a demo account, display the original default portal data.
13. Allow the student to move between dashboard modules.
14. End when the student signs out or closes the application.

Flowchart



11. Structured Programming Application

The project demonstrates structured programming by breaking the program into smaller functions. Each function performs a specific task, making the system easier to read, maintain, test, and improve.

Variables and Constants

The program uses variables such as `current_user`, `login_view`, `cal_yr`, `cal_mo`, and `signup_vars`. It also uses constants such as `STUDENTS_FILE` and the color dictionary `C`.

Data Types

- String: names, passwords, emails, course names.
- Integer: assignment IDs, calendar dates, progress values, scores.
- Boolean: terms agreement, urgent status, unread messages.
- List: students, courses, assignments, conversations.
- Dictionary: each student, course, assignment, and color configuration.

Decision Structures

The code uses `if`, `elif`, and `else` statements to validate input, check login credentials, decide whether a user has custom subjects, determine grades, and display error or success messages.

Loops

The system uses loops to display courses, assignments, navigation buttons, demo accounts, calendar events, and messages. Loops reduce repeated code and allow the interface to handle multiple records.

User-Defined Functions

Function	Explanation
<code>load_students()</code>	Loads saved student records from the JSON file.
<code>save_students()</code>	Saves the current student list to the JSON file.
<code>generate_student_id()</code>	Creates a unique ID for a new student.
<code>parse_subjects()</code>	Converts comma-separated or new-line subject text into a clean subject list.
<code>build_courses_from_subjects()</code>	Creates course dictionaries from subjects entered during signup.
<code>get_user_courses()</code>	Returns default courses for demo accounts or custom courses for new students.
<code>get_user_assignments()</code>	Creates assignment data based on the current student subjects.
<code>signup_submit()</code>	Validates signup data and creates a new student account.
<code>login()</code>	Checks the student ID and password and opens the dashboard.
<code>navigate()</code>	Switches between dashboard pages.

12. Functional Requirements

- The system shall allow a student to register with personal details.
- The system shall require students to enter at least one subject during signup.
- The system shall automatically generate a student ID when one is not entered.
- The system shall allow registered students to log in.
- The system shall reject invalid login credentials.
- The system shall display a dashboard after successful login.
- The system shall display courses based on the student account type.
- The system shall display assignments and deadlines.
- The system shall display grades and progress.
- The system shall display calendar events.
- The system shall display lecturer messages.
- The system shall allow the user to sign out.

13. Non-Functional Requirements

Requirement	Explanation
Usability	The interface is clean, simple, and easy to navigate.
Reliability	Student data is saved in JSON so it can be reused.
Maintainability	Functions separate different tasks such as signup, login, rendering, and data processing.
Performance	The program uses local file storage and loads quickly.
Accessibility	Labels, buttons, and clear text are used to guide the user.
Security	The password field is hidden during input; the email password is kept outside the source code using an environment variable in the final package.

14. Data Privacy and Ethics

The system collects basic student information such as name, email, password, institution, and subjects. The data is used only for login and academic display inside the application. Ethical use of data means that student records should not be shared without permission.

For privacy improvement, the final package does not keep the email application password directly in the code. Instead, it uses an environment variable named `EMAIL_APP_PASSWORD`. This prevents the project from exposing sensitive email credentials when uploaded to GitHub.

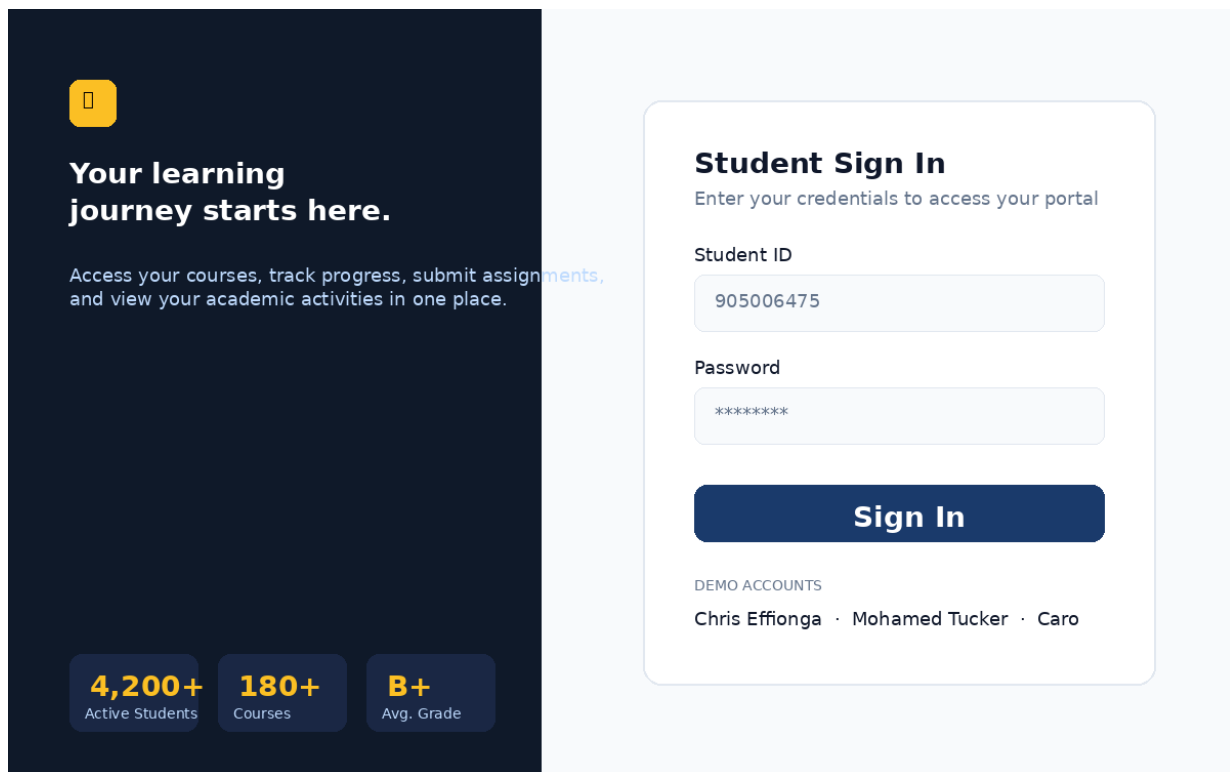
15. Testing and Results

Test Case	Input	Expected Result	Actual Result	Status
Signup with empty	Blank first name	Error message	Error message	Pass

first name			displayed	
Signup with invalid email	chrismail.com	Invalid email error	Invalid email error displayed	Pass
Signup without subjects	No subject entered	Missing subjects error	Missing subjects error displayed	Pass
Signup with valid data	Name, email, subjects, password	Account created	Account created and saved	Pass
Login with valid demo account	905006475 / chris123	Dashboard opens	Dashboard opens with default courses	Pass
Login with new student	Generated ID / password	Dashboard opens	Dashboard opens with custom subjects	Pass
Courses page	Open courses	Subject cards displayed	Student subjects displayed	Pass
Assignments page	Open assignments	Assignments displayed	Assignments generated from subjects	Pass
Wrong login	Wrong password	Login error	Invalid login message displayed	Pass

16. Screenshots

Login Page



The screenshot displays the login interface of a Student Academic Portal. On the left, a dark blue sidebar contains a yellow mobile app icon, a welcome message, a description of portal features, and three statistics: 4,200+ Active Students, 180+ Courses, and a B+ Avg. Grade. The main area on the right is light blue and contains a 'Student Sign In' form with fields for Student ID (905006475) and Password (masked with asterisks), a 'Sign In' button, and a section for demo accounts: Chris Effiong, Mohamed Tucker, and Caro.

Your learning journey starts here.

Access your courses, track progress, submit assignments, and view your academic activities in one place.

4,200+
Active Students

180+
Courses

B+
Avg. Grade

Student Sign In
Enter your credentials to access your portal

Student ID
905006475

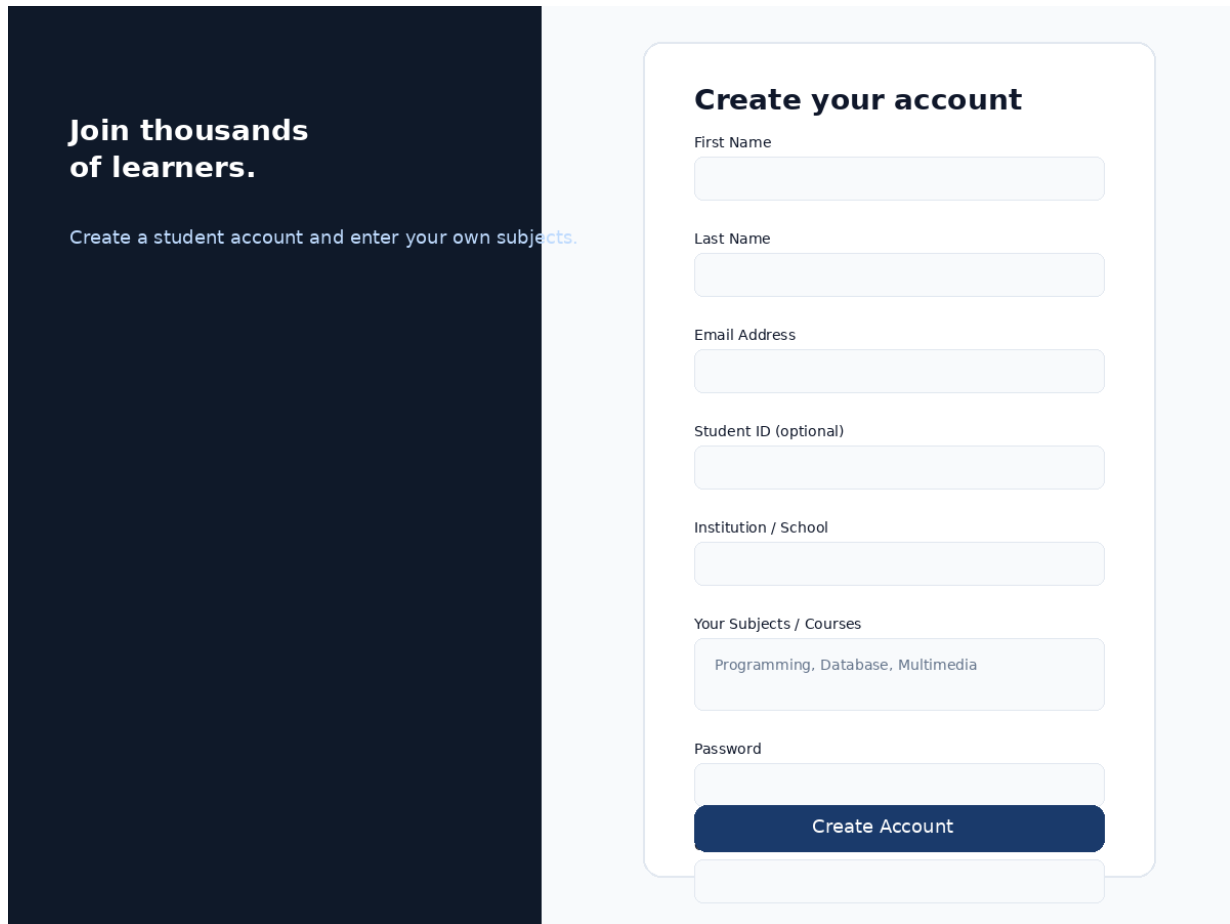
Password

Sign In

DEMO ACCOUNTS
Chris Effiong · Mohamed Tucker · Caro

Figure: Login Page of the Student Academic Portal.

Signup Page with Subject Entry



The image shows a web page for creating a student account. On the left, a dark blue vertical banner contains the text "Join thousands of learners." and "Create a student account and enter your own subjects." in white. On the right, a light gray rounded rectangle contains the "Create your account" form. The form has several input fields: "First Name", "Last Name", "Email Address", "Student ID (optional)", and "Institution / School". Below these is a section for "Your Subjects / Courses" with a text input field containing "Programming, Database, Multimedia". At the bottom of the form is a "Password" field and a dark blue "Create Account" button. A light gray horizontal bar is located below the button.

Join thousands of learners.

Create a student account and enter your own subjects.

Create your account

First Name

Last Name

Email Address

Student ID (optional)

Institution / School

Your Subjects / Courses

Programming, Database, Multimedia

Password

Create Account

Figure: Signup Page with Subject Entry of the Student Academic Portal.

Dashboard Page

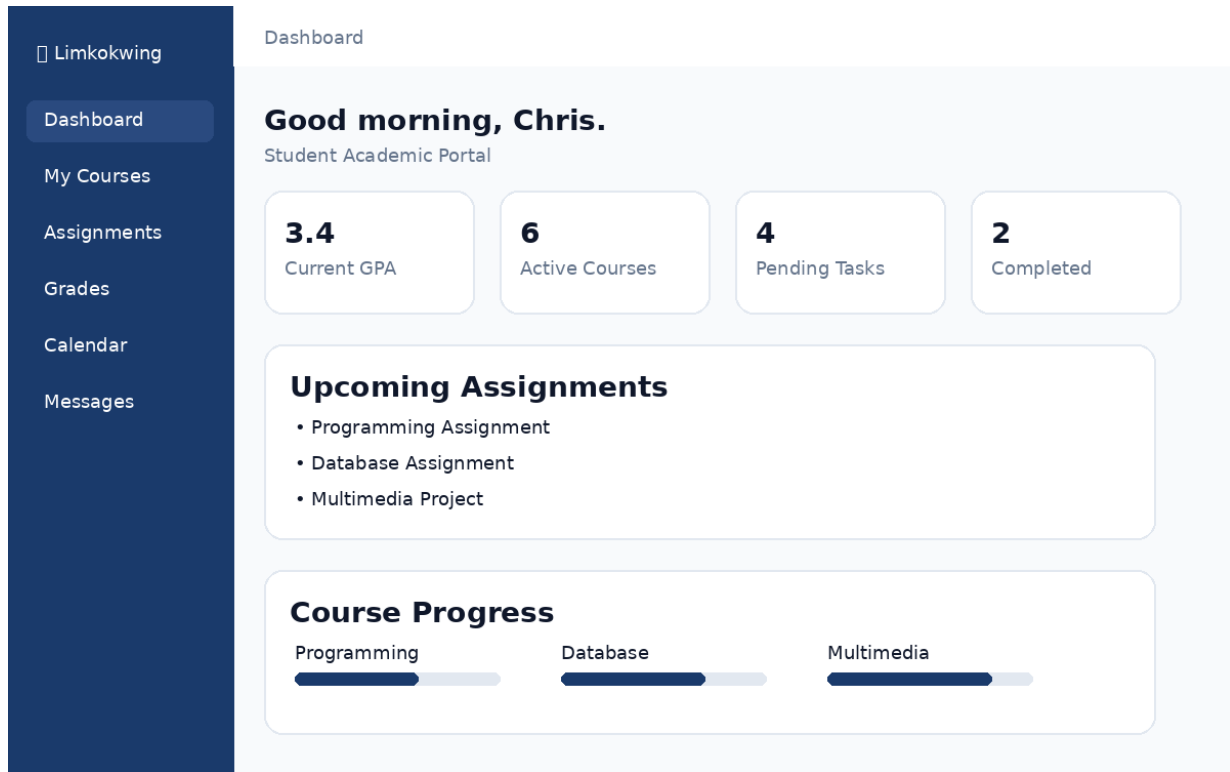


Figure: Dashboard Page of the Student Academic Portal.

Courses Page



Figure: Courses Page of the Student Academic Portal.

17. GitHub and Open-Source Evidence

The project should be placed in a GitHub repository named PROG103_FinalProject_Class_GroupName. The repository should contain the Python source files, README.md, documentation, screenshots, and license file.

Recommended Git Commands

```
git init
git status
git add .
git commit -m "Initial commit: Student Academic Portal"
git branch -M main
git remote add origin <your-github-repository-url>
git push -u origin main
```

Open-Source License

The project uses the MIT License because it is a simple and common open-source license that allows other students and developers to use, modify, and share the project with proper attribution.

18. Installation and User Guide

Installation Steps

15. Install Python 3.10 or above.
16. Extract the project ZIP folder.
17. Open the folder in VS Code or another Python IDE.
18. Open a terminal in the project folder.
19. Run: `pip install -r requirements.txt`
20. Run: `python now.py`

Demo Login Accounts

Student Name	Student ID	Password
Chris Effiong	905006475	chris123
Mohamed Tucker	905007895	Moh123
Caro	905006789	Caro123

How to Register a New Student

21. Click “Don’t have an account? Sign up”.
22. Enter first name, last name, email, institution, subjects, password, and confirm password.
23. Subjects can be separated using commas or new lines.
24. Agree to the terms checkbox.
25. Click Create Account.
26. Save the generated student ID and use it to sign in.

19. Challenges Encountered

- Designing a clean GUI with many pages.
- Keeping demo account data different from new student data.
- Generating courses, assignments, calendar events, and messages from user-entered subjects.
- Saving records in JSON and loading them again.
- Managing global state for the current user and current page.
- Handling optional email sending without exposing sensitive credentials.

20. Conclusion and Recommendations

The Student Academic Portal Management System successfully meets the requirements of the PROG103 final project. It is a GUI-based Python application that solves an education-related problem in Sierra Leone and aligns with SDG 4: Quality Education. The system demonstrates structured programming through functions, variables, data types, decision structures, loops, file handling, and modular design.

Recommendations

- Add password hashing for stronger login security.
- Connect the system to MySQL for larger data storage.
- Create an admin dashboard for lecturers and school staff.
- Add PDF report card generation.
- Add real assignment upload and download features.
- Add OTP verification for password reset.

21. References

Python Software Foundation. (2026). Python documentation.

CustomTkinter Documentation. (2026). CustomTkinter widgets and interface guide.

United Nations. (2026). Sustainable Development Goal 4: Quality Education.

Limkokwing University of Creative Technology Sierra Leone. (2026). PROG103 Principle of Structured Programming Project Brief.

Matplotlib Documentation. (2026). Embedding charts in Tkinter applications.

22. Appendix: Key Source Code Explanation

Subject Parsing Function

```
def parse_subjects(subject_text):  
    raw = subject_text.replace("\n", ",").split(",")  
    subjects = []  
    for item in raw:  
        subject = item.strip()  
        if subject and subject.lower() not in [s.lower() for s in subjects]:  
            subjects.append(subject)  
    return subjects
```

This function converts the subject text entered during signup into a clean list. It accepts both commas and new lines and removes repeated subjects.

Custom Course Builder

```
def build_courses_from_subjects(subjects):
    courses = []
    for i, subject in enumerate(subjects):
        courses.append({"name": subject, "code": f"SUB-{101 + i}", "teacher": "Assigned Lecturer"})
    return courses
```

This function uses a loop to generate course records for every subject entered by a student.

Demo Account Protection

```
def get_user_courses():
    if current_user and current_user.get("subjects"):
        return build_courses_from_subjects(current_user["subjects"])
    return COURSES
```

This function ensures that newly registered students see their own subjects, while demo accounts continue using the original default courses.